SRI GURU TEGH BAHADUR INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY

Department of BCA

SYNOPSIS REPORT

ON

CUSTOMER CHURNSENSE AI SYSTEM USING ML

Submitted in partial fulfilment of the requirement for
Software Training Report (STR), BCA – Vth Semester

Submitted by:

GURSIMRAN KAUR

Roll No.: 02490202024

Section: BCA 5A

Under the Guidance of:

Coordinator: Ms. Nehu Gumber

HOD (BCA): Dr. Raj Kumar

A. Title of the Project

Customer ChurnSense AI System Using Machine Learning

Customer churn is a common problem faced by businesses when customers discontinue their services or stop maintaining their relationship with the company. For example, a customer may cancel a subscription or move to a competing service provider.

The proposed project, **Customer ChurnSense AI System**, uses Machine Learning to analyse customer-related information and predict the possibility of customer churn. The system uses factors such as tenure, monthly charges, total charges, contract type and service-related information to identify patterns in customer behaviour.

The purpose of the system is to provide an early prediction of whether a customer is likely to leave or continue with the company. This can help businesses take suitable action towards customer retention before the customer actually leaves.

B. Statement about the Problem

Customer churn is a significant concern for businesses that provide subscription-based or continuous services. Industries such as telecommunications, banking, DTH, SaaS and e-commerce can lose considerable revenue when existing customers discontinue their services.

A major difficulty is that companies often become aware of customer churn only after the customer has already cancelled the service or switched to another provider. At that point, it becomes difficult to take any preventive action.

The main problem addressed by this project is to determine whether historical customer information can be used to predict the likelihood of a customer leaving the company.

The system considers information such as customer tenure, monthly charges, contract type and service usage, and uses these attributes to identify patterns associated with customer churn.

The central objective is therefore to develop a Machine Learning model that can analyse previous customer data and predict which customers may be more likely to discontinue the service. Such predictions can give businesses an opportunity to take timely measures for customer retention.

C. Why the Particular Topic is Chosen? (Present State of the Art)

Present State of the Art

Customer retention has become an important area of concern for modern businesses because maintaining existing customers is essential for long-term business growth. With customers having multiple alternatives available in today's competitive market, businesses need better ways to understand and predict customer behaviour.

Machine Learning is increasingly being used for predictive analysis because it can identify patterns from historical data and use those patterns to make predictions.

The topic was selected for the following reasons:

- Customer churn is a practical problem faced by several industries, including telecom, banking and OTT/streaming services.
- Machine Learning provides a way to predict customer behaviour before the actual churn takes place.
- Traditional rule-based or statistical approaches may not be sufficient when several customer characteristics need to be considered together. Machine Learning classification algorithms can identify relationships between multiple attributes.
- Churn prediction is a suitable real-world application of a classification problem and provides an opportunity to implement the complete Machine Learning process.
- The project allows the application of concepts studied during the course, including data preprocessing, categorical encoding, feature scaling, model training, evaluation and model persistence.
- Logistic Regression has been selected because it is comparatively simple, interpretable and suitable for classification problems.

Therefore, this topic provides a combination of a practical business problem and the application of the Machine Learning concepts studied as part of the course.

D. Objective and Scope of the Project

Objectives

The main objectives of the project are:

- To analyse the customer dataset and understand the different factors that may influence customer churn.
- To identify customer characteristics and patterns that can help in predicting whether a customer is likely to leave or continue with the company.
- To preprocess the available customer data so that it can be effectively used for Machine Learning.
- To develop and train a Logistic Regression classification model to predict customer churn, where 1 represents a customer who is likely to churn and 0 represents a customer who is likely to stay.
- To evaluate the performance of the developed model using accuracy, confusion matrix and classification report.

- To save the trained Machine Learning model and scaler using pickle, allowing them to be reused for future predictions.

- To develop a simple real-time prediction module that accepts customer information and provides a prediction of whether the customer is likely to churn or stay.

Scope

The project focuses on a structured customer dataset containing information such as customer ID, gender, age, tenure, monthly charges, total charges, contract type, internet service and churn status.

The system treats customer churn as a binary classification problem, where the model predicts:

- 0 – Customer is likely to stay

- 1 – Customer is likely to churn

The project uses supervised Machine Learning to learn patterns from historical customer data and apply them to new customer records.

Although the current dataset focuses on customer service-related information, the same approach can be applied to other subscription-based and service-based industries such as telecommunications, banking, OTT platforms, insurance, gyms and SaaS, wherever historical customer data is available.

The current scope is limited to analysing the available dataset, developing the Logistic Regression model, evaluating its performance and using the saved model for real-time customer churn prediction.

E. Analysis, Design, Development & Testing Methodology

1. Analysis

The customer churn dataset is first examined to understand its structure and characteristics. Functions such as data.shape and data.info() are used to identify the number of records, columns and data types. data.describe() is used to obtain a statistical summary of the numerical features.

2. Design – Data Preprocessing

The dataset is prepared before applying the Machine Learning model through the following steps:

- Missing Value Treatment: Missing values in numerical columns are replaced with the respective column mean using fillna().

- Categorical Encoding: Categorical features such as Gender and Contract are converted into numerical values using Label Encoding.

- Exploratory Data Analysis: A count plot is used to understand the churn distribution, while a correlation heatmap is used to examine relationships between features.

- Feature and Target Separation: The input features are separated into X, while the target variable, Churn, is stored in y.

- Train-Test Split: The data is divided into 80% training data and 20% testing data using train_test_split().

- Feature Scaling: StandardScaler is applied to bring the numerical features to a similar scale.

3. Development – Model Building

A Logistic Regression model is used because the project involves a binary classification problem. The model is simple, interpretable and suitable for predicting whether a customer is likely to churn or stay.

The model is trained using:

model.fit(x_train, y_train)

After training, it can be used to predict the churn status of test and new customer records.

4. Testing

The trained model is tested using the unseen test data with model.predict(x_test). Its performance is measured using accuracy score, confusion matrix and classification report, which provides precision, recall, F1-score and support.

The trained model and StandardScaler are then saved using pickle so they can be loaded later for real-time predictions.

f. Hardware and Software to be Used

Category	Details
Operating System	Windows 11
Programming Language	Python 3.14
IDE / Platform	Google Colab / VS code/render
Libraries	NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Pickle
Hardware	Intel i3 (or higher) processor, 4 GB RAM (or higher), 20 GB free disk space

G. Testing Technologies to be Used

The following methods will be used for testing and evaluating the system:

- **Train-Test Split (80:20):** Used to evaluate the model on unseen data.

- **Accuracy Score:** Measures the overall percentage of correct predictions.
- **Confusion Matrix:** Shows the correct and incorrect classifications made by the model.
- **Classification Report:** Provides Precision, Recall, F1-score and Support.
- **Manual Real-Time Testing:** Sample customer records are provided to the saved model and scaler to verify that the prediction module works correctly.

H. What Contribution / Value Addition Would the Project Make

The Customer ChurnSense system provides businesses with an early indication of customers who may be likely to leave.

Its main contributions are:

- Helps identify potentially at-risk customers before they churn.
- Supports timely retention actions such as suitable offers or discounts.
- Provides a reusable saved model and scaler for future predictions.
- Demonstrates a complete Machine Learning workflow from preprocessing to prediction.
- Can be adapted for other service-based and subscription-based businesses.

The system therefore demonstrates how customer data can be used to support more informed customer retention decisions.

I. Limitations / Constraints of the Project

The project has the following limitations:

- The model is trained on a relatively small, static dataset of approximately 1,000 records, so it may not generalise perfectly to a larger or different customer base.
- Only Logistic Regression is currently used. Other algorithms may provide better results.
- The system works with a batch/offline dataset and is not connected to a live company database or CRM system.
- The available dataset contains a limited number of customer attributes.
- Label Encoding is used for categorical variables, although One-Hot Encoding may be more appropriate for some nominal categories.

J. Conclusion, Future Scope for Modification

Conclusion

The **Customer ChurnSense AI System Using Machine Learning** demonstrates an end-to-end approach to customer churn prediction. The project covers data analysis, preprocessing, encoding, feature scaling, model training and evaluation.

A Logistic Regression model is trained to classify customers based on their available information, and its performance is evaluated using standard classification metrics. The model and scaler are also saved using pickle so that they can be reused for new customer predictions.

The project provides a practical example of how Machine Learning can help businesses identify potential customer churn and take timely retention measures.

Future Scope

The system can be further improved by:

- Comparing Logistic Regression with **Random Forest, Decision Tree, SVM and XGBoost**.
- Applying hyperparameter tuning and cross-validation.
- Deploying the model as a web application using **Flask or Streamlit**.
- Connecting the system with a live or regularly updated company database.
- Using **One-Hot Encoding** and additional features such as customer complaints and usage patterns.

K. References and Bibliography

- Scikit-learn Documentation – https://scikit-learn.org/stable/documentation.html
- Pandas Documentation – https://pandas.pydata.org/docs/
- NumPy Documentation – https://numpy.org/doc/
- Matplotlib Documentation – https://matplotlib.org/stable/contents.html
- Seaborn Documentation – https://seaborn.pydata.org/
- Kaggle – Telco Customer Churn Dataset – https://www.kaggle.com/
- Python Machine Learning by Sebastian Raschka – Reference Book

FRONTEND CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Customer Churn Prediction</title>
    <style>
        :root{
            --bg: #0b1620;
            --card:#0f2230;
            --text:#e6f2f8;
            --muted:#9fb7c6;
            --primary:#17a2b8;
            --primary-dark:#0e7580;
            --input-bg:#0b1722;
            --input-border:#274051;
            --success-bg:rgba(16,185,129,0.12);
            --success-border:#0f766e;
            --success-text:#d1fae5;
            --danger-bg:rgba(239,68,68,0.08);
            --danger-border:#ef4444;
            --danger-text:#ffd7d7;
        }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: var(--bg);
            margin: 0;
            padding: 0;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            color: var(--text);
        }
        .container {
            background-color: var(--card);
            padding: 30px;
            border-radius: 12px;
            box-shadow: 0 8px 30px rgba(2,6,23,0.6);
            width: 100%;
            max-width: 900px;
            margin: 30px;
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
            gap: 16px;
        }
        .form-group {
            display: flex;
            flex-direction: column;
        }
        label {
            margin-bottom: 6px;
            color: var(--muted);
            font-weight: 600;
            font-size: 14px;
            display: flex;
            align-items: baseline;
            gap: 8px;
        }
        .range {
            font-size: 12px;
            color: var(--muted);
            font-weight: 400;
        }
        input, select {
            padding: 10px;
            border: 1px solid var(--input-border);
            border-radius: 6px;
            font-size: 14px;
            transition: border-color 0.18s, background-color 0.18s;
            background-color: var(--input-bg);
            color: var(--text);
        }
        input::placeholder{ color: rgba(255,255,255,0.18); }
        input:focus, select:focus {
            border-color: var(--primary);
            outline: none;
            box-shadow: 0 0 0 3px rgba(23,162,184,0.08);
```

```
<html lang="en">
<head>
    <style>
        #result {
            font-size: 18px;
            font-weight: bold;
            border-radius: 6px;
            display: none;
        }
        .success {
            background-color: var(--success-bg);
            color: var(--text);
            border: 1px solid var(--success-border);
        }
        .danger {
            background-color: var(--danger-bg);
            color: var(--text);
            border: 1px solid var(--danger-border);
        }
        .invalid {
            border-color: var(--danger-border) !important;
            box-shadow: 0 0 0 3px rgba(239,68,68,0.06);
        }
        .err-msg {
            color: var(--danger-text);
            font-size: 12px;
            margin-top: 6px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Customer Churn Prediction</h1>
        <form id="prediction-form">
            <div class="form-grid">
                <div class="form-group"><label>State</label>
                    <select id="State" name="State" required>
                        {% for s in state_options %}
                        <option value="{{ s }}">{{ s }}</option>
                        {% endfor %}
                    </select>
```

```
<html lang="en">
<body>
    <div class="container">
        <form id="prediction-form">
            <div class="form-grid">
                    </select>
                </div>
                <div class="form-group"><label>Account Length <span class="range">{% if limits.get('Account Length') %}{{ '%.2f' % li
                    <input type="number" id="Account Length" name="Account Length" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Area Code <span class="range">{% if limits.get('Area Code') %}{{ '%.2f' % limits['Area
                    <input type="number" id="Area Code" name="Area Code" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Phone</label><input type="text" id="Phone" name="Phone" value="0" placeholder="0" requ
                <div class="form-group"><label>Int'l Plan</label><select id="Int'l Plan" name="Int'l Plan"><option value="no">No</opt
                <div class="form-group"><label>VMail Plan</label><select id="VMail Plan" name="VMail Plan"><option value="yes">Yes</o
                <div class="form-group"><label>VMail Message <span class="range">{% if limits.get('VMail Message') %}{{ '%.2f' % limi
                    <input type="number" id="VMail Message" name="VMail Message" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Day Mins <span class="range">{% if limits.get('Day Mins') %}{{ '%.2f' % limits['Day Mi
                    <input type="number" step="0.01" id="Day Mins" name="Day Mins" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Day Calls <span class="range">{% if limits.get('Day Calls') %}{{ '%.2f' % limits['Day
                    <input type="number" id="Day Calls" name="Day Calls" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Day Charge <span class="range">{% if limits.get('Day Charge') %}{{ '%.2f' % limits['Da
                    <input type="number" step="0.01" id="Day Charge" name="Day Charge" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Eve Mins</label><input type="number" step="0.01" id="Eve Mins" name="Eve Mins" value="
                <div class="form-group"><label>Eve Calls <span class="range">{% if limits.get('Eve Calls') %}{{ '%.2f' % limits['Eve
                    <input type="number" id="Eve Calls" name="Eve Calls" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Eve Charge <span class="range">{% if limits.get('Eve Charge') %}{{ '%.2f' % limits['Ev
                    <input type="number" step="0.01" id="Eve Charge" name="Eve Charge" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Night Mins <span class="range">{% if limits.get('Night Mins') %}{{ '%.2f' % limits['Ni
                    <input type="number" step="0.01" id="Night Mins" name="Night Mins" value="0" placeholder="0" required>
                </div>
                <div class="form-group"><label>Night Calls <span class="range">{% if limits.get('Night Calls') %}{{ '%.2f' % limits['
                    <input type="number" id="Night Calls" name="Night Calls" value="0" placeholder="0" required>
```

```
}

function clearFieldError(inputEl) {
    inputEl.classList.remove('invalid');
    const msg = inputEl.parentElement.querySelector('.err-msg');
    if (msg) msg.textContent = '';
}

function validateField(inputEl) {
    const name = inputEl.name;
    if (!limits || !limits[name]) {
        clearFieldError(inputEl);
        return true;
    }
    const lim = limits[name];
    const raw = inputEl.value;
    if (raw === null || raw === undefined || raw.toString().trim() === '') {
        showFieldError(inputEl, 'Value required');
        return false;
    }
    const num = Number(raw);
    if (isNaN(num)) {
        showFieldError(inputEl, 'Must be a number');
        return false;
    }
    if (lim.min !== undefined && num < lim.min) {
        showFieldError(inputEl, `Minimum allowed is ${lim.min.toFixed(2)}`);
        return false;
    }
    if (lim.max !== undefined && num > lim.max) {
        showFieldError(inputEl, `Maximum allowed is ${lim.max.toFixed(2)}`);
        return false;
    }
    clearFieldError(inputEl);
    return true;
}
```

```
// Attach listeners to inputs that have corresponding limits
document.querySelectorAll('#prediction-form input, #prediction-form select').forEach(el => {
    el.addEventListener('input', () => validateField(el));
});

document.getElementById('prediction-form').addEventListener('submit', async function(e) {
    e.preventDefault();

    const formEl = this;
    const inputs = Array.from(formEl.querySelectorAll('input, select'));
    let ok = true;
    inputs.forEach(inp => {
        if (!validateField(inp)) ok = false;
    });
    const resultDiv = document.getElementById('result');
    resultDiv.style.display = 'none';
    resultDiv.className = '';
    if (!ok) {
        resultDiv.textContent = 'Please fix input errors highlighted in red.';
        resultDiv.classList.add('danger');
        resultDiv.style.display = 'block';
        return;
    }

    const formData = new FormData(this);
    const data = {};
    formData.forEach((value, key) => {
        data[key] = isNaN(value) || value.trim() === '' ? value : Number(value);
    });

    try {
        const response = await fetch('/predict', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });
        const resData = await response.json();
```



```
            });

            try {
                const response = await fetch('/predict', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify(data)
                });
                const resData = await response.json();

                if (resData.error) {
                    resultDiv.textContent = 'Error: ' + resData.error;
                    resultDiv.classList.add('danger');
                } else {
                    if(resData.prediction === 1) {
                        resultDiv.textContent = 'Prediction: Customer WILL Churn 🔴';
                        resultDiv.classList.add('danger');
                    } else {
                        resultDiv.textContent = 'Prediction: Customer will NOT Churn ✅';
                        resultDiv.classList.add('success');
                    }
                }
                resultDiv.style.display = 'block';
            } catch (error) {
                console.error('Error:', error);
                resultDiv.textContent = 'An error occurred while predicting.';
                resultDiv.classList.add('danger');
                resultDiv.style.display = 'block';
            }
        });
    </script>
</body>
</html>
```

BACKEND CODE:

```python
from flask import Flask, request, jsonify, render_template
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
import warnings

warnings.filterwarnings('ignore')

app = Flask(__name__)

# Load and prepare the model
data = pd.read_csv('churn.csv')

# Also load raw copy for template-driven UI (preserve original values)
raw = pd.read_csv('churn.csv')

# Handle missing values just like in the notebook
data.fillna(data.mean(numeric_only=True), inplace=True)

# Prepare UI choices and numeric limits from raw data
raw.fillna(raw.mean(numeric_only=True), inplace=True)
try:
    state_options = sorted(raw['State'].dropna().unique().tolist())
except Exception:
    state_options = []

from pandas.api import types as pdtypes
limits = {}
for col in raw.columns:
    if col == 'Churn?':
        continue
    try:
        if pdtypes.is_numeric_dtype(raw[col]):
            col_min = raw[col].min()
            col_max = raw[col].max()
            # Cast to int when both are integers
            if float(col_min).is_integer() and float(col_max).is_integer():
                col_min = int(col_min)
                col_max = int(col_max)
            limits[col] = {'min': col_min, 'max': col_max}
```

```python
# Encode categorical features
import pandas.api.types as ptypes
encoders = {}
for col in data.columns:
    if (data[col].dtype == 'object' or ptypes.is_string_dtype(data[col])) and col != 'Churn?':
        le = LabelEncoder()
        data[col] = le.fit_transform(data[col].astype(str))
        encoders[col] = le

# Encode target variable
target_le = LabelEncoder()
data['Churn?'] = target_le.fit_transform(data['Churn?'].astype(str))

# Prepare features and target
X = data.drop('Churn?', axis=1)
y = data['Churn?']

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train the Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_scaled, y)

@app.route('/')
def index():
    return render_template('index.html', state_options=state_options, limits=limits)

@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Accept either JSON body or form-encoded data
        req_data = request.get_json(silent=True)
        if not req_data:
            req_data = request.form.to_dict()

        input_df = pd.DataFrame([req_data])

        # Ensure correct column order and include any missing columns
        input_df = input_df.reindex(columns=X.columns)
```

```
    input_df = pd.DataFrame([req_data])

    # Ensure correct column order and include any missing columns
    input_df = input_df.reindex(columns=X.columns)

    # Encode categorical inputs from the web form
    for col in input_df.columns:
        # Fill missing values: categorical -> most common known class, numeric -> median
        if input_df[col].isnull().any():
            if col in encoders:
                input_df[col].fillna(encoders[col].classes_[0], inplace=True)
            else:
                # fallback to median of training data for numeric columns
                try:
                    input_df[col] = input_df[col].fillna(X[col].median())
                except Exception:
                    input_df[col].fillna(0, inplace=True)

        if col in encoders:
            le = encoders[col]
            known_classes = set([str(c) for c in le.classes_])
            # Handle unseen labels by mapping to the first known class
            input_df[col] = input_df[col].apply(lambda x: str(x) if str(x) in known_classes else str(le.classes_[0]))
            input_df[col] = le.transform(input_df[col].astype(str))

    # Scale inputs
    input_scaled = scaler.transform(input_df)

    # Predict
    prediction = model.predict(input_scaled)

    # Map prediction back to original class label
    result_class = target_le.inverse_transform(prediction)[0]

    # Check if the result indicates churn (adjusting for 'True.' string format in original data)
    is_churn = 1 if 'True' in str(result_class) else 0

    return jsonify({'prediction': is_churn})
```

CERTIFICATE:

